

Subgrid and Container Layouts Report

Parker Woolsey

June 14, 2026

Site Details

My live advanced layouts pages are at <https://pwoolls1.github.io/WDD331R/unit-4/grid-layouts/advanced/sticky-demo.html> and the source code is at <https://pwoolls1.github.io/WDD331R/index.html>.

Reflection

Container Queries vs. Media Queries

Container queries are the right choice when a component needs to respond to the size of its own parent instead of the entire browser window. For example, if I have a card component that could appear in a wide main content area or a narrow sidebar, a container query makes more sense. The card can change its layout based on the space it actually has, not just the viewport size.

Media queries are still better for larger page-level layout changes. For example, switching an entire page from a single-column mobile layout to a two-column desktop layout is still a good use for a media query. In that case, the overall viewport size matters because the whole page structure is changing.

The main difference is context. Container queries are component-focused, while media queries are page-focused.

The Sticky Overflow Bug

A sticky element works by staying in normal document flow until it reaches a certain scroll position, then it sticks relative to its nearest scrolling ancestor. The problem happens when a parent element has overflow set to something like hidden, auto, or scroll. At the browser level, that ancestor becomes the scroll container, so the sticky element is no longer sticking relative to the page like I might expect. It is limited by that parent's scroll area.

If sticky was failing, I would inspect the element in DevTools and check each parent container going upward. I would look for any ancestor with overflow: hidden, overflow: auto, or overflow: scroll. Once I found it, I would either remove that overflow rule, move the sticky element outside of that container, or restructure the layout so the sticky item has the correct scroll context.

Subgrid Tradeoffs

I would ship subgrid on a production site today, but only with a fallback. Subgrid is useful because it solves alignment problems cleanly, especially when cards have different amounts of content but still need their internal sections to line up. It keeps the layout more consistent without needing extra fixed heights or awkward spacing tricks.

However, I would not rely on subgrid without checking browser support for the client's audience. My fallback strategy would be to create a regular grid or flexbox version first that still looks good, even if the alignment is not as perfect. Then I would use @supports to add subgrid only for browsers that support it.

That way, users with modern browsers get the cleaner aligned version, while everyone else still gets a functional and readable layout.